

Búsqueda Heurística (1a. parte)

Javier Béjar, Javier Vázquez

Algoritmos Básicos de la Inteligencia Artificial - 2022/2023 1Q

CS - GIA



Búsqueda Informada

- ⦿ Supone la existencia de una función de evaluación que se utiliza para guiar el proceso de búsqueda
- ⦿ En cada momento se selecciona el estado o las operaciones más prometedores según esa función

- ⦿ Algoritmos que trabajan en el espacio de estados:
 - Branch and Bound, Best First Search
 - A*, IDA*, MA*, ...
- ⦿ Algoritmos que trabajan en el espacio de soluciones:
 - Hill climbing
 - Simulated annealing
 - Algoritmos Genéticos

Búsqueda Informada guiada por un coste (G)

Búsqueda Informada guiada por un coste (G)

- ⦿ Supone la existencia de una función de evaluación que debe estimar el coste mínimo acumulado desde un estado del espacio de estados al estado inicial ($g(n)$)
- ⦿ Esta función de evaluación a priori parece fácil de calcular (sumatorio de costes acumulados desde el estado inicial al nodo actual), pero veremos que no es tan fácil calcular el coste del camino mínimo.
- ⦿ La función se utiliza para guiar el proceso haciendo que en cada momento se seleccione el estado o las operaciones más prometedores
- ⦿ Algunos algoritmos no siempre garantizan encontrar una solución (de existir ésta)
- ⦿ Algunos algoritmos no siempre garantizan encontrar la solución óptima (la que tiene un coste acumulado menor)

Ramificación y poda

- ⊙ Se guarda para cada estado el coste (hasta el momento) de llegar desde el estado inicial a dicho estado.
 - Guarda el coste mínimo global hasta el momento
 - Pero es una estimación del coste del camino mínimo: puede existir otro nodo aun por expandir que proporcione un camino de coste menor al nodo inicial.
- ⊙ Deja de explorar una rama cuando su coste es mayor que el mínimo actual
- ⊙ Si el coste de los nodos es uniforme equivale a una búsqueda por niveles: generaliza BFS, DFS.
- ⊙ Existen múltiples implementaciones que usan diferentes estrategias para explorar el espacio de estado (orden de generación de sucesores, orden de selección).

6

Algoritmo: Branch and Bound v1 - Depth First Bounded-Cost Search (limite: entero)

```

Est_abiertos.insertar(Estado inicial)
Actual ← Est_abiertos.primer() ; Solucion ← null
mientras no Est_abiertos.vacia?() hacer
    Est_abiertos.borrar_primer(); Est_cerrados.insertar(Actual)
    si es_final?(Actual) entonces
        si coste_acumulado (Actual) ≤ limite entonces
            limite ← coste_acumulado (Actual)
            Solucion ← Actual
        en otro caso
            hijos ← generar_sucesores_con_poda (Actual, limite)
            hijos ← tratar_repetidos (Hijos, Est_cerrados, Est_abiertos)
            Est_abiertos.insertar(Hijos)
    Actual ← Est_abiertos.primer()
  
```

- ⊙ La estructura de abiertos es una pila, se escoge el siguiente nodo por profundidad prioritaria
- ⊙ La función de generación de sucesores poda aquellos con coste acumulado mayor al límite
- ⊙ Cuando el nodo actual es solución se ajusta el límite de poda y sigue buscando.

7

Características

- ⊙ El uso de memoria es mínimo (como en el DFS) si no se guardan los nodos cerrados ($O(rp)$)
- ⊙ La complejidad temporal es como un DFS, ($O(r^p)$) pero expande menos nodos gracias a la poda.
- ⊙ Encuentra la solución de coste óptimo si y solo si está dentro del límite de poda.
- ⊙ No podemos parar en la primera solución encontrada, hay que buscar todas las soluciones dentro del límite de poda y quedarse la mejor
- ⊙ Si el coste de los nodos es uniforme equivale a una búsqueda por profundidad limitada.
- ⊙ Problema: requiere una buena elección del límite de coste.

8

Algoritmo: Branch and Bound v2 - Uniform Cost Search

Est_abiertos.insertar(Estado inicial)

Actual ← Est_abiertos.primer()

mientras no es_final?(Actual) y no Est_abiertos.vacía?() hacer

| Est_abiertos.borrar_primer()

| Est_cerrados.insertar(Actual)

| hijos ← generar_sucesores (Actual)

| hijos ← tratar_repetidos (Hijos, Est_cerrados, Est_abiertos)

| Est_abiertos.insertar(Hijos)

 | Actual ← Est_abiertos.primer()

- ⊙ La estructura de abiertos es una cola con prioridad
- ⊙ La prioridad la marca la función de coste (coste acumulado del camino desde la raíz)
- ⊙ En cada iteración se escoge el nodo de la frontera de coste menor (el primero de la cola)
- ⊙ Cuando el nodo actual es solución acaba, y se garantiza que es la solución óptima en coste.

9

Características

- ⦿ No requiere definir un límite de coste
- ⦿ La complejidad temporal es como un BFS ($O(r^p)$), pero expande menos nodos gracias a la poda.
- ⦿ Encuentra la solución de coste óptimo si y solo si está dentro del límite de poda.
- ⦿ Si el coste de los nodos es uniforme equivale a una búsqueda por profundidad limitada.
- ⦿ Problema: requiere mantener todos los nodos (abiertos y cerrados) en memoria ($O(r^p)$).

Búsqueda Heurística guiada por un
estimador (H)

- ⊙ Supone la existencia de una función de evaluación que debe estimar la distancia desde un estado del espacio de estados al (a un) objetivo ($h(n)$)
- ⊙ Esta función de evaluación es una heurística, que ha de estimar la distancia (sin calcularla)
- ⊙ La función se utiliza para guiar el proceso haciendo que en cada momento se seleccione el estado o las operaciones más prometedores
- ⊙ No siempre se garantiza encontrar una solución (de existir ésta)
- ⊙ No siempre se garantiza encontrar la solución más próxima (la que se encuentra a una distancia, número de operaciones menor)

12

Algoritmo: Greedy Best First

```
Est_abiertos.insertar(Estado inicial)
```

```
Actual ← Est_abiertos.primer()
```

```
mientras no es_final?(Actual) y no Est_abiertos.vacía?() hacer
```

```
    Est_abiertos.borrar_primer()
```

```
    Est_cerrados.insertar(Actual)
```

```
    hijos ← generar_sucesores (Actual)
```

```
    hijos ← tratar_repetidos (Hijos, Est_cerrados, Est_abiertos)
```

```
    Est_abiertos.insertar(Hijos)
```

```
    Actual ← Est_abiertos.primer()
```

- ⊙ La estructura de abiertos es una cola con prioridad
- ⊙ La prioridad la marca la función de estimación (coste del camino que falta hasta la solución)
- ⊙ En cada iteración se escoge el nodo mas cercano a la solución (el primero de la cola), esto provoca que no se garantice la solución óptima

13

Operaciones:

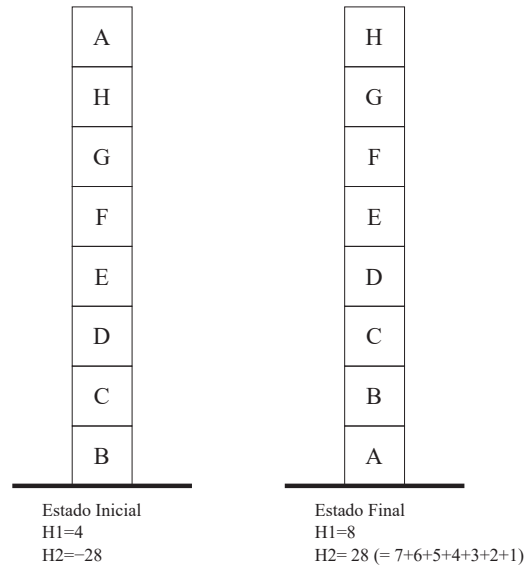
- situar un bloque libre en la mesa
- situar un bloque libre sobre otro bloque libre

Heurístico 1:

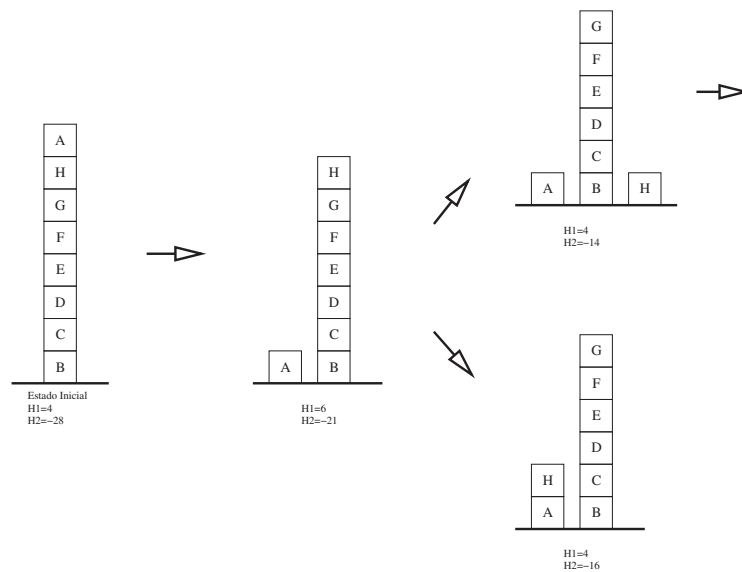
- sumar 1 por cada bloque que esté colocado sobre el bloque que debe
- restar 1 si el bloque no está colocado sobre el que debe

Heurístico 2:

- si la estructura de apoyo es correcta sumar 1 por cada bloque de dicha estructura
- si la estructura de apoyo no es correcta restar 1 por cada bloque de dicha estructura



14



15

2	8	3
1	6	4
7		5

Estado Inicial

1	2	3
8		4
7	6	5

Estado Final

Posibles heurísticos (estimadores del coste a la solución)

- ⊙ $h(n) = w(n) = \# \text{desclasificados}$
- ⊙ $h(n) = p(n) = \text{suma de distancias a la posición final}$
- ⊙ $h(n) = p(n) + 3 \cdot s(n)$
 donde $s(n)$ se obtiene recorriendo las posiciones no centrales y si una ficha no va seguida por su sucesora sumar 2, si hay ficha en el centro sumar 1

16

- ⊙ Es una implementación del Best First recursiva con coste lineal en espacio $O(rp)$
- ⊙ **Olvida** una rama cuando su coste supera la mejor alternativa
- ⊙ El coste de la rama olvidada se almacena en el padre como su nuevo coste
- ⊙ La rama es reexpandida si su coste vuelve a ser el mejor (regeneramos toda la rama olvidada)

17

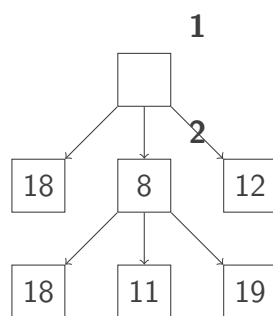
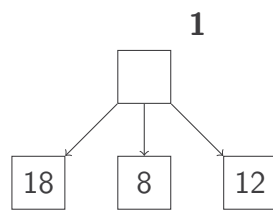
Procedimiento: BFS-recursivo (nodo,c_alternativo,ref nuevo_coste,ref solucion)

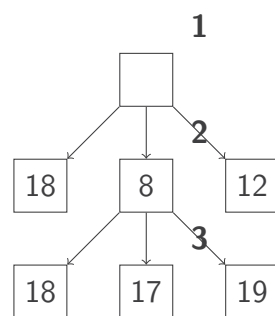
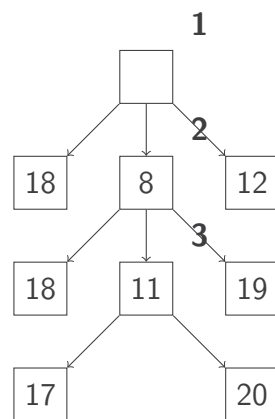
```

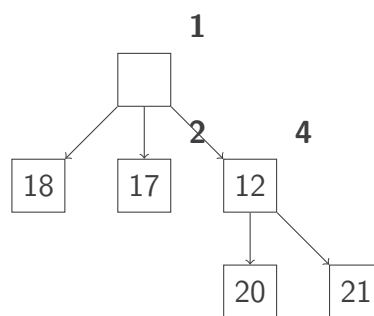
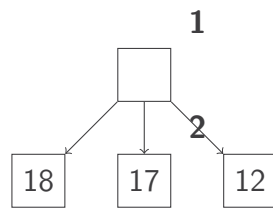
si es_solucion?(nodo) entonces
  | solucion.añadir(nodo)
en otro caso
  | sucesores ← generar_sucesores (nodo)
  | si sucesores.vacio?() entonces
  | | nuevo_coste ←  $+\infty$ ; solucion.vacio()
  | en otro caso
  | | fin ← falso
  | | mientras no fin hacer
  | | | mejor ← sucesores.mejor_nodo()
  | | | si mejor.coste() > c_alternativo entonces
  | | | | fin ← cierto; solucion.vacio(); nuevo_coste ← mejor.coste()
  | | | en otro caso
  | | | | segundo ← sucesores.segundo_mejor_nodo()
  | | | | BFS-recursivo(mejor,min(c_alternativo,segundo.coste()),nuevo_coste, solucion)
  | | | | si solucion.vacio?() entonces
  | | | | | mejor.coste(nuevo_coste)
  | | | | en otro caso
  | | | | | solucion.añadir(mejor); fin ← cierto

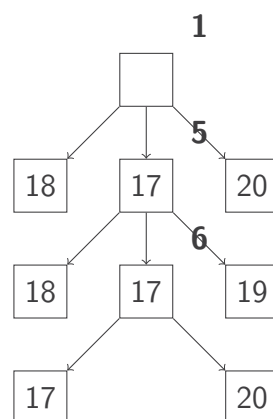
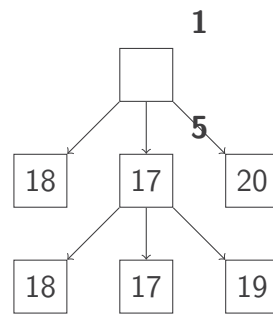
```

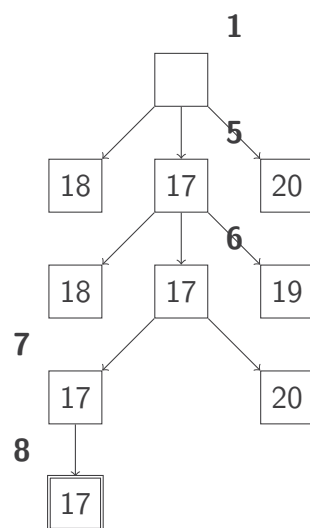
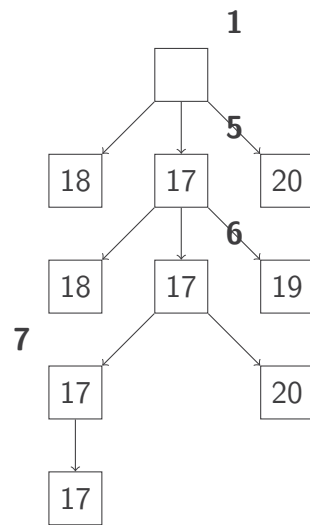












- ⦿ El límite de memoria (lineal en espacio) provoca muchas reexpansiones en ciertos problemas
- ⦿ Al no poder controlar los repetidos su coste en tiempo puede elevarse si hay ciclos
- ⦿ Solución: Relajar la restricción de memoria